Lab 12 — Speedup y eficiencia

CC3069 Computación Paralela y Distribuida · Universidad del Valle de Guatemala
Diego López #23747, Alejandro Antón #####

1. Datos del equipo y condiciones de medición

Dato	**Valor**
Modelo de CPU	AMD Ryzen 7 8845HS w/ Radeon 780M Graphics
Núcleos físicos / lógicos	8 / 16 (lscpu: 8 cores por socket, 2 threads por core; Windows: 8 / 16)
Núcleos de rendimiento / eficiencia	No se distinguen: lscpu muestra 8 núcleos con la misma L2 (8 instancias de 1 MiB). Desde WSL no se puede verificar el tipo de núcleo, así que no lo afirmamos.
Hyper-Threading / SMT	Sí, 2 hilos por núcleo
Caché	L1d 32 KiB y L2 1 MiB por núcleo; L3 16 MiB compartida
pmax	16 (omp_get_num_procs)
Sistema	Windows 11 + WSL2 Ubuntu 24.04.1 LTS, kernel 6.6.87.2-microsoft-standard-WSL2 (lscpu: hipervisor Microsoft)
Compilador	gcc 13.3.0, flags -O2 -Wall -fopenmp (Makefile del starter, sin warnings)

Condiciones: laptop conectada a la corriente, Docker Desktop cerrado (distro docker-desktop en estado Stopped), sin ventanas de navegador abiertas (solo procesos de Edge en segundo plano). OneDrive quedó abierto con la sincronización pausada y VS Code en ejecución. Los binarios y CSV se generaron en ~/lab12 (sistema de archivos de WSL) y luego se copiaron al repositorio. Cada medición es la mediana de 5 repeticiones (REPS = 5 en el programa). Las predicciones se escribieron a las 18:23 y la primera medición empezó a las 18:24.

2. Parte 1 — La parte que no se puede repartir (Amdahl)

Predicción (antes de correr)

Modelo S(p) = 1 / ((1 − fp) + fp/p) con pmax = 16: fp = 1.00 → 16.00; 0.95 → 9.14; 0.90 → 6.40; 0.75 → 3.37; 0.50 → 1.88.

- fp = 1.00: esperábamos S(16) entre 13 y 15.5 (E 0.81–0.97). Los hilos 9 a 16 son el segundo hilo SMT de cada núcleo; como el trabajo es una cadena de operaciones dependientes, SMT debería ayudar bastante pero no como un núcleo completo. Además el turbo baja con todos los núcleos activos y WSL2 agrega virtualización.
- fp = 0.90: esperábamos S(16) entre 5.8 y 6.4, un poco debajo del modelo (6.40) por overhead de OpenMP y frecuencia.

Contraste con lo medido

fp = 1.00: medimos S(16) = 13.203 (E = 0.825), en el extremo bajo del rango. Hasta p = 15 la curva fue casi ideal (S(15) = 14.004, E = 0.934), pero con p = 16 el speedup bajó. fp = 0.90: medimos S(16) = 5.621, por debajo del rango; con p = 15 fue 5.864, que sí cae dentro. El fallo de la predicción fue no anticipar la caída en p = 16.

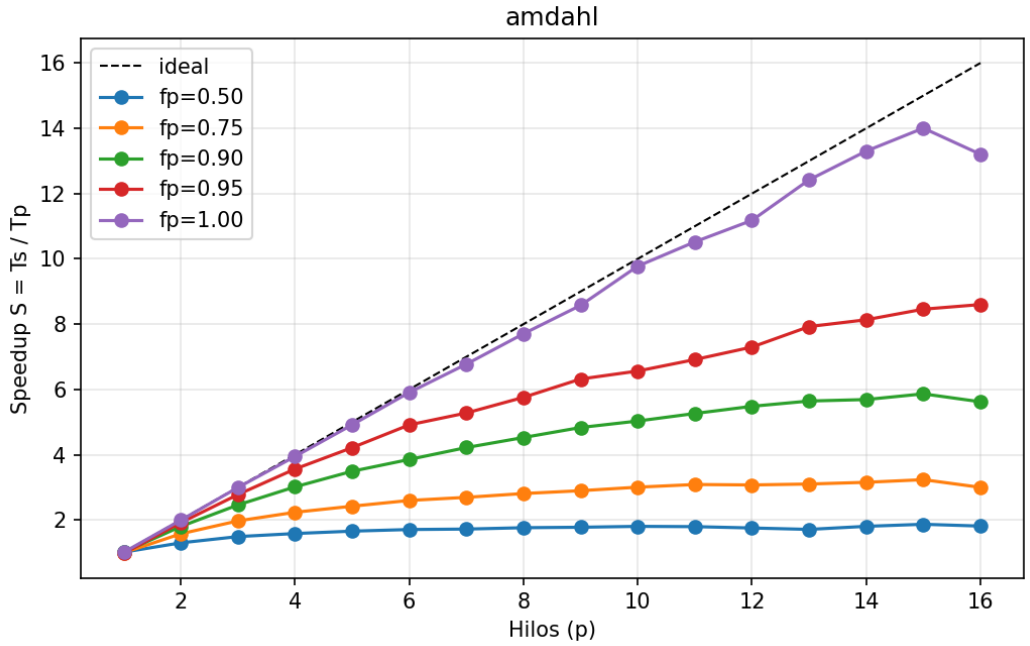


Tabla oficial (pmax = 16)

fp	S (p=2)	S (p=4)	S (pmax)	E (pmax)	p con F máxima
1.00	1.995	3.947	13.203	0.825	15
0.95	1.916	3.562	8.600	0.537	13
0.90	1.800	3.015	5.621	0.351	9
0.75	1.577	2.236	3.000	0.187	3
0.50	1.300	1.585	1.813	0.113	1

Cálculo a mano de dos filas (fp = 0.90)

```
Fila 1: p = 2     t_seq = 0.497038 s   t_par = 0.276077 s
  S = Ts/Tp = 0.497038 / 0.276077 = 1.800
  E = S/p   = 1.800 / 2 = 0.900
  C = p·Tp  = 2 · 0.276077 = 0.552154 hilo-s
```

```
  F = S/C   = 1.800 / 0.552154 = 3.261

Fila 2: p = 16    t_seq = 0.497038 s   t_par = 0.088429 s
  S = Ts/Tp = 0.497038 / 0.088429 = 5.621
  E = S/p   = 5.621 / 16 = 0.351
  C = p·Tp  = 16 · 0.088429 = 1.414864 hilo-s
  F = S/C   = 5.621 / 1.414864 = 3.973
```

Los mismos valores aparecen en amdahl_con_S_E.csv generado por graficar.py.

Preguntas

P1.1. No llegamos a S = p: con fp = 1.00, S(16) = 13.203 y E(16) = 0.825. Hasta p = 8 (un hilo por núcleo físico) la eficiencia se mantuvo entre 0.96 y 1.00 (S(8) = 7.705). Causas que proponemos:

- Hyper-Threading (SMT): solo hay 8 núcleos físicos. De p = 8 a p = 15 los 7 hilos extra compartieron núcleo y sumaron 14.004 − 7.705 = 6.30 de speedup, unos 0.9 por hilo: SMT ayudó mucho porque la cadena dependiente deja unidades libres, pero no rinde como un núcleo completo. No encaja en las cuatro fugas.
- Ningún CPU libre con p = 16: con 16 hilos ocupamos todos los CPU lógicos y Windows, WSL2 y VS Code siguen necesitando tiempo de CPU. Si el sistema interrumpe un hilo, los demás lo esperan en la barrera final: S cayó de 14.004 (p = 15) a 13.203 (p = 16). Se ve como desbalance (η_b), pero su origen está fuera del programa. Es una hipótesis que no medimos directamente; el mismo salto aparece en Gustafson (Parte 4).
- Costo de crear y sincronizar hilos (η_c): existe, pero cada región paralela dura ~0.5 s, así que pesa poco (E(2) = 0.997).
- Turbo boost: con más núcleos activos la frecuencia por núcleo puede bajar. No la medimos; como E(2) = 0.997, el efecto con pocos hilos es pequeño. No encaja en las cuatro fugas.

P1.2. Con fp = 0.75 la curva es cóncava y se aplana hacia el límite 1/(1 − 0.75) = 4: S(2) = 1.577, S(4) = 2.236, S(8) = 2.810, S(15) = 3.235 (el modelo da 1.600, 2.286, 2.909 y 3.333). El hilo 2 aporta ΔS = +0.582 y el hilo 8 solo ΔS = +0.117, unas 5 veces menos. Cada hilo nuevo solo acelera la parte paralela, que ya es una fracción cada vez más pequeña del tiempo total; el 25 % secuencial no se mueve.

P1.3. Modelo. La fracción fp de T1 se reparte entre p hilos y el resto no:

```
Tp = T1·(1 − fp) + T1·fp/p
S(p) = T1/Tp = T1 / [T1·((1 − fp) + fp/p)] = 1 / ((1 − fp) + fp/p)
```

T1 se cancela: el speedup solo depende de fp y de p.

P1.4. Con fp = 0.90 y p = 16: S = 1/(0.10 + 0.90/16) = 1/0.15625 = 6.400. Medido: 5.621, error relativo (5.621 − 6.400)/6.400 = −12.2 %. Parte de esa distancia es la caída de p = 16: con p = 15 el modelo da 6.250 y medimos 5.864 (−6.2 %). El modelo no incluye overhead de OpenMP, SMT ni la interferencia del sistema.

P1.5. Cuando p → ∞, fp/p → 0 y S → 1/(1 − fp). Con fp = 0.90 el máximo es 10, aunque tuviéramos 1000 núcleos: S(1000) = 1/(0.10 + 0.0009) = 9.911.

P1.6. De 1/S = 1 − fp·(1 − 1/p) se despeja fp = (p/(p − 1))·(1 − 1/S). Con S(16) = 5.621: fp = (16/15)·(1 − 1/5.621) = 0.8769, menor que 0.90 (con p = 15: 0.8888). La diferencia es tiempo que el modelo cuenta como secuencial pero que en realidad es overhead de crear, sincronizar y combinar el reduction: la fuga η_c. En p = 16 también entra la interferencia del sistema descrita en P1.1.

P1.7. Con F = S/C = Ts/(p·Tp²) y el modelo, minimizar p·Tp² da p* = fp/(1 − fp). Para fp = 0.90, p* = 9, y medimos F máxima justo en p = 9 (F = 5.222). El menor tiempo no está en el mismo p: está en p = 15 (Tp = 0.084759 s). De 9 a 15 hilos el tiempo sigue bajando, pero cada hilo extra rinde menos y F cae. En el caso de 16 números de clase el óptimo fue p = 4 porque la fase de combinación crece con $\log_2 p$ y el trabajo era mínimo; aquí la parte secuencial es el 10 % y no hay una fase que crezca con p, por eso el óptimo es mayor.

fp	p* teórico = fp/(1−fp)	p con F máxima medido
0.50	1	1
0.75	3	3
0.90	9	9
0.95	19 (> pmax, se esperaría 16)	13
1.00	sin máximo (se esperaría 16)	15

Para 0.95 y 1.00 el máximo medido quedó antes de pmax por la pérdida de eficiencia en p = 14–16.

P1.8. Con 20 s totales y 2 s secuenciales, 1 − fp = 0.10 y fp = 0.90:

```
S(64) = 1/(0.10 + 0.90/64) = 8.77      Tp = 20/8.77 = 2.28 s
E = 8.77/64 = 0.137 (13.7 %)            C = 64·2.28 = 146 hilo-s
F = S/C = 8.77/146 = 0.060              (secuencial: F = 1/20 = 0.050)
Límite con infinitos núcleos: S ≤ 10, o sea Tp ≥ 2 s.
p = 9:  S = 5.00  Tp = 4.00 s  C = 36 hilo-s  F = 0.139  (F máxima, p* = 9)
p = 16: S = 6.40  Tp = 3.13 s  C = 50 hilo-s  F = 0.128
```

Respuesta al jefe: con 64 núcleos pagaría 146 hilo-segundos para ganar 8.8×, aprovechando el 13.7 % de la máquina. Con 9 a 16 núcleos ya obtiene entre 5× y 6.4× con mucho mejor efectividad. El cuello de botella son los 2 s de lectura: si bajaran a 0.5 s (programa de 18.5 s), 1 − fp = 0.027 y el límite subiría a ~37×.

3. Parte 2 — ¿Vale la pena paralelizar algo pequeño?

2a. El Ts honesto

P2.1. En amdahl_fp100.csv, fila p = 1: t_seq = 0.495953 s y t_par = 0.491456 s. La versión OpenMP con 1 hilo fue 0.91 % más rápida. No es un efecto sistemático: en los otros archivos la diferencia con p = 1 fue +2.09 % (fp 0.95), −1.52 % (0.90), +0.55 % (0.75) y −1.35 % (0.50). Con trabajos de ~0.5 s, el overhead de OpenMP con un hilo queda dentro del ruido de ±2 %.

P2.2. Si Tp(1) > Ts (lo normal), S′ = Tp(1)/Tp(p) = S·Tp(1)/Ts sale inflado. Es tramposo porque la referencia debe ser el mejor programa secuencial; usar el paralelo con 1 hilo mete el overhead de paralelizar en la base y lo esconde. En nuestros datos Tp(1) fue algo menor que Ts, así que S′(16) = 13.084 contra el honesto 13.203: aquí habría salido un poco desinflado, pero el problema de fondo es el mismo.

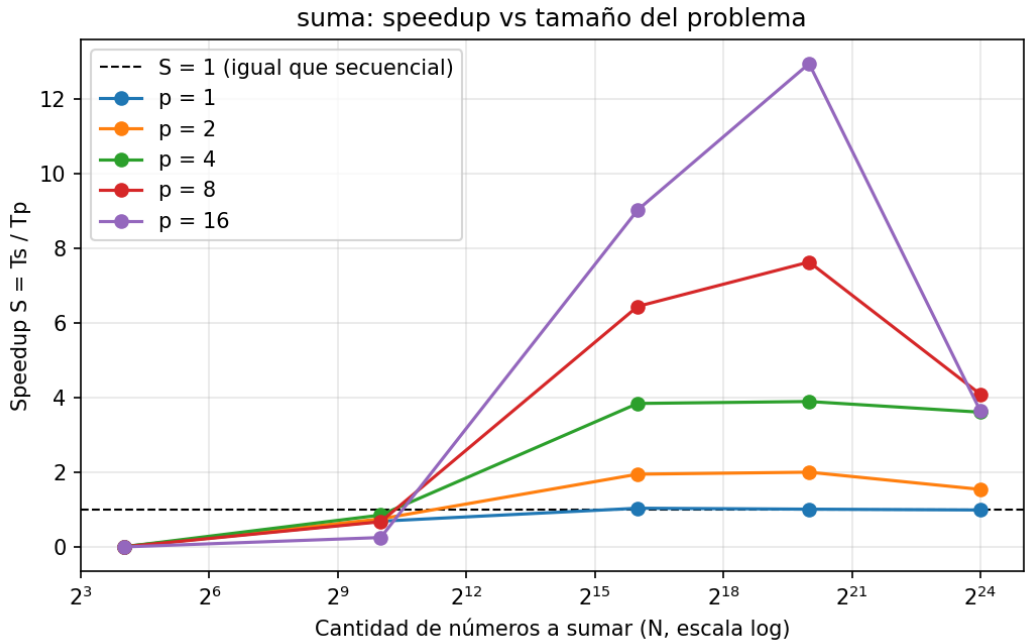
2b. Sumar N números

Predicción (antes de correr)

Cada parallel for cuesta del orden de 1–2 µs y sumar un elemento del orden de 1 ns. Con N = 16 la versión paralela sería cientos de veces más lenta. Esperábamos S < 1 en N = 1024 y S > 1 desde N = 65536. Con N = 1M (8 MiB, cabe en L3) S cerca de p; con N = 16M (128 MiB) S lejos de 16 por ancho de banda de memoria.

Contraste

Se cumplió: S < 1 en N = 1024 para todo p y S > 1 desde N = 65536 (S(16) = 9.029). N = 16 fue 747 veces más lento. Con N = 1M, S(16) = 12.946, y con N = 16M, S(16) = 3.638. Lo que no anticipamos es que con N = 1024 y p = 16 el speedup (0.253) fuera peor que con p = 4 (0.855): más hilos cuestan más overhead.



N	p=1	p=2	p=4	p=8	p=16
16	0.013	0.005	0.005	0.004	0.001
1024	0.690	0.753	0.855	0.676	0.253
65536	1.038	1.950	3.844	6.444	9.029
1048576	1.013	2.003	3.896	7.632	12.946
16777216	0.991	1.542	3.610	4.090	3.638

P2.3. Con N = 16 y p = 16: S = t_seq/t_par = 3.0×10^{-9} / 2.240×10^{-6} = 0.0013. La versión paralela es ~747 veces más lenta. (t_seq se imprime con resolución de 1 ns, así que el factor real está entre ~640 y ~900.)

P2.4. El modelo de dos fases supone que cada suma y cada combinación cuestan una unidad y que despertar hilos, repartir, sincronizar y combinar es gratis. En la máquina esos costos son microsegundos y dominan sobre 16 sumas de ~1 ns. Es la fuga η_c (comunicación y sincronización).

P2.5. En cada parallel for OpenMP tiene que:

- despertar a los hilos del equipo y esperar que arranquen;
- calcular el reparto de iteraciones (schedule static: un rango contiguo por hilo);
- crear una copia privada de s por hilo e inicializarla en 0;
- cada hilo suma su pedazo;
- combinar las sumas parciales en s, con sincronización;
- esperar en la barrera implícita del final y devolver el control al hilo maestro.

Con N = 16 el trabajo útil son ~3 ns; todo lo demás es overhead: t_par = 0.236 µs con 1 hilo y 2.240 µs con 16. El costo crece con p porque hay más hilos que despertar y sincronizar.

P2.6. Con Tp ≈ O + c·N/p y Ts = c·N, la versión paralela conviene cuando N > N* = O / (c·(1 − 1/p)). Con O ≈ t_par(N = 16, p = 16) = 2.240 µs y c = t_seq/N = 6.54×10^{-10} s por elemento (medido en N = 65536): N* ≈ 3653. El modelo coincide con los datos: para N = 1024 predice S ≈ 0.28 (medido 0.253) y para N = 65536 predice ≈ 8.7 (medido 9.029). En la rejilla, el primer N con S > 1 fue 65536 para todo p. Regla práctica: un parallel for vale la pena cuando el ciclo secuencial tarda mucho más (10× a 100×) que el overhead de OpenMP, aquí ~2 µs; es decir, desde decenas de miles de elementos de trabajo liviano.

P2.7. No. Con N = 16 777 216 y p = 16, S = 3.638 y E = 0.227; además se estanca: S(4) = 3.610, S(8) = 4.090, S(16) = 3.638. El arreglo ocupa 128 MiB y no cabe en la L3 de 16 MiB, así que los datos vienen de la RAM. El recurso compartido es el ancho de banda de memoria: el secuencial lee a ~12.1 GB/s, con p = 8 se llega a ~49.4 GB/s y con p = 16 a ~43.9 GB/s; más hilos no leen más rápido cuando el bus ya está saturado. Con N = 1M (8 MiB, cabe en L3) sí llegamos a S(16) = 12.946. No encaja limpiamente en las cuatro fugas: es saturación de un recurso compartido.

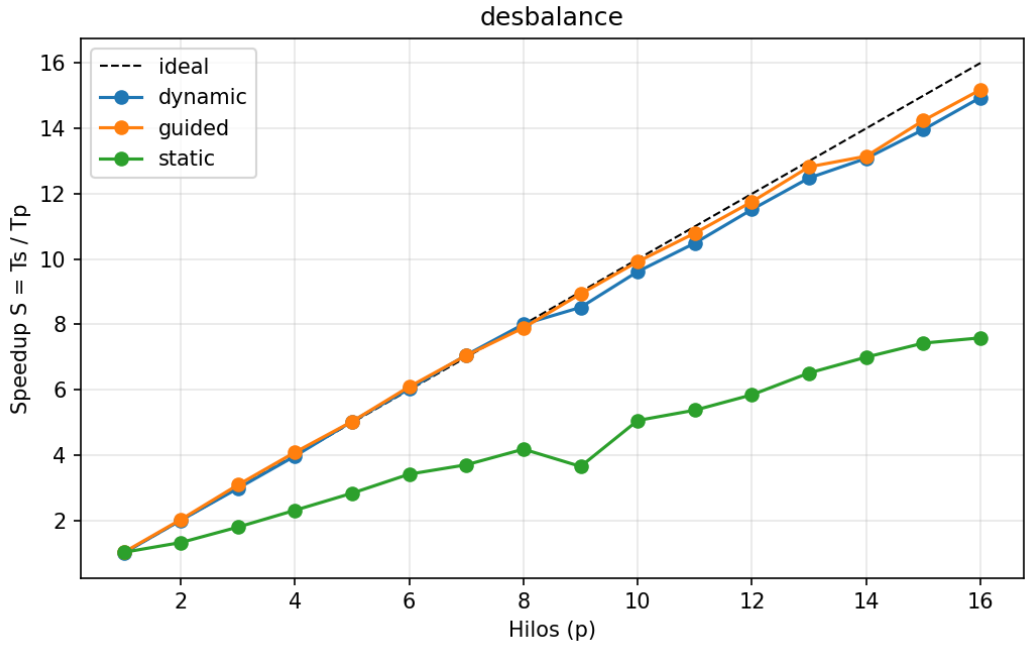
4. Parte 3 — Cuando a un hilo le toca más trabajo

Predicción (antes de correr)

costo(i) = 1 + 2000·i/W crece como una recta; el trabajo total es ≈ W·1001. Con static el bloque k tiene costo promedio 1 + 2000·(k + 0.5)/p. Para p = 4 el último hilo recibe (1 + 2000·3.5/4)/(4·1001) = 1751/4004 = 0.437 del trabajo, así que S ≈ 1/0.437 = 2.29. Para p = 16 static, S ≈ 8.3 como máximo. Esperábamos que dynamic y guided superaran claramente a static.

Contraste

Medimos S(4) = 2.317 con static (+1.3 % respecto a 2.29): la predicción del trapecio fue muy buena. Con p = 16 static dio 7.591, debajo del máximo de 8.26 que predijimos, y dynamic/guided llegaron a 14.942 y 15.181.



P3.1. S(16): static = 7.591, dynamic = 14.942, guided = 15.181. Gana guided: supera a static por ΔS = +7.590 (+100 %, el doble) y a dynamic por ΔS = +0.238 (+1.6 %, diferencia pequeña que puede ser ruido). dynamic supera a static por +7.352 (+96.9 %).

P3.2. Con static cada hilo recibe un bloque contiguo de W/p iteraciones; el trabajo de un bloque es el área de un trapecio bajo la recta costo(x) = 1 + 2000·x/W. El más cargado es el último:

```
p = 2: último bloque [W/2, W], costo de 1001 a 2001, promedio 1501
       fracción = (W/2 · 1501) / (W · 1001) = 1501/2002 = 0.7498
       S predicho = 1/0.7498 = 1.334      S medido = 1.329  (−0.4 %)
p = 4: último bloque [3W/4, W], costo de 1501 a 2001, promedio 1751
       fracción = (W/4 · 1751) / (W · 1001) = 1751/4004 = 0.4373
       S predicho = 1/0.4373 = 2.287      S medido = 2.317  (+1.3 %)
```

Sumando exactamente costo(i) del programa (con su redondeo entero) se obtiene 0.7499 y 0.4374, prácticamente lo mismo. El speedup con static queda limitado por el hilo que recibe el bloque más caro.

P3.3. Mientras el último hilo termina su bloque caro, los primeros ya acabaron y esperan sin hacer nada en la barrera implícita del final del parallel for. Eso es tiempo pagado y no usado, y se ve como baja eficiencia. El modelo predice E = 1/(p·f):

p	E predicho	E medido (static)	Tiempo de espera aprox.
2	0.667	0.664	34 %
4	0.572	0.579	42 %
8	0.533	0.523	48 %
16	0.516	0.474	53 %

Cuando p crece, E tiende a 1001/2001 ≈ 0.50: la mitad del tiempo pagado se va en espera. Con dynamic y guided E se mantuvo entre 0.93 y 1.0 (dynamic E(16) = 0.934, guided 0.949). Hubo una anomalía en static con p = 9: el modelo con bloques de ~W/9 predice S ≈ 4.77 y medimos 3.657 (E = 0.406). Nuestra hipótesis, no verificada, es que con 9 hilos en 8 núcleos dos hilos comparten núcleo por SMT y, si uno de ellos tiene un bloque caro, tarda más.

P3.4. dynamic decide el reparto en tiempo de ejecución: cada vez que un hilo se desocupa tiene que tomar el siguiente bloque de un contador compartido, lo que requiere una operación atómica o un candado. Aquí ese costo no se nota: hay W/64 = 625 bloques y cada uno cuesta ~64 · 1001 · 1.27 ns ≈ 81 µs de cálculo, de modo que con p = 1 dynamic quedó a −0.53 % del secuencial (ruido). Pero con iteraciones livianas, como la suma de la Parte 2 (~0.65 ns por elemento), un bloque de 64 iteraciones son ~42 ns de trabajo y el costo de pedir cada bloque pesaría tanto como el trabajo: es el mismo overhead de sincronización (η_c) de la Parte 2. Además dynamic pierde la localidad de bloques contiguos y el reparto deja de ser reproducible. Conviene cuando el costo por iteración varía y cada iteración es pesada; si el costo es uniforme, static no paga nada en tiempo de ejecución.

P3.5. Nuestra CPU no permite verificar núcleos de rendimiento y eficiencia desde WSL (lscpu muestra 8 núcleos con la misma L2). En una CPU híbrida, static da el mismo número de iteraciones a cada hilo, pero un núcleo de eficiencia las ejecuta más lento que uno de rendimiento: aunque el costo por iteración sea idéntico, el tiempo por hilo no lo es y los hilos rápidos esperan al lento. En nuestro equipo pasa algo parecido con SMT: con p entre 9 y 15 algunos hilos comparten núcleo y corren más lento que los que tienen un núcleo para ellos solos.

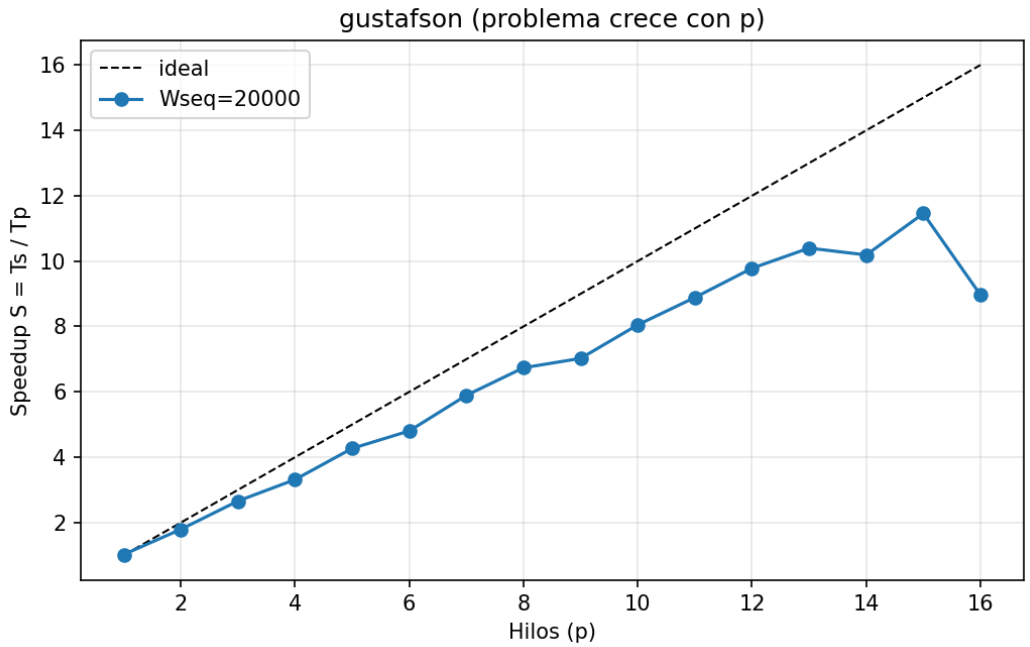
5. Parte 4 — ¿Y si el problema crece con los núcleos?

Predicción (antes de correr)

Con p = 1 la parte secuencial es 20000/120000 = 1/6, así que fp ≈ 0.83. Con la fórmula de la Parte 1 el límite es 1/(1 − 5/6) = 6 y S(16) = 1/(1/6 + (5/6)/16) = 4.57. Creímos que ese límite no aplicaría aquí: el trabajo paralelo crece con p mientras la parte secuencial queda fija, así que t_par casi no cambiaría y t_seq crecería. Esperábamos S ≈ (20000 + 100000·16)/120000 = 13.5 en p = 16, algo menos por SMT.

Contraste

Acertamos en lo cualitativo: el límite de 6 se superó desde p = 8 (S = 6.737) y llegamos a S(15) = 11.457. Quedamos por debajo de 13.5: la eficiencia bajó a ~0.80 desde p = 9 (SMT) y en p = 16 hubo una caída fuerte (S = 8.966).



P4.1. t_seq crece casi lineal con p: de 0.152 s (p = 1) a 2.064 s (p = 16), ×13.55, porque es el tiempo de resolver en un solo núcleo un problema de 20000 + 100000·p unidades. t_par casi no se mueve: de 0.149 s a 0.169 s en p = 15 (×1.14), porque cada hilo siempre hace sus 100000 unidades y la parte secuencial siempre es de 20000. En p = 16 t_par saltó a 0.230 s (×1.55), el mismo salto de p = 16 que vimos en la Parte 1.

P4.2. Amdahl con fp = 5/6 predice S(16) = 4.571 y un límite de 6; medimos S(16) = 8.966 y S(15) = 11.457. Es tan distinto porque Amdahl supone un problema fijo: ahí fp no cambia. Aquí la parte secuencial es fija pero el problema crece, así que la fracción secuencial del trabajo total baja con p (en p = 16 es 20000/1 620 000 = 1.2 %). El código es el mismo; lo que cambió es la pregunta que se mide.

P4.3. s = t_parte_seq / t_par por fila:

p	**t_seq (s)**	**t_par (s)**	**t_parte_seq (s)**	**s**	**S medido**	**S = s + (1−s)·p**	**error**
1	0.152328	0.148848	0.025392	0.1706	1.023	1.000	+2.3 %
2	0.272636	0.152012	0.025548	0.1681	1.794	1.832	−2.1 %
3	0.398348	0.149760	0.025326	0.1691	2.660	2.662	−0.1 %
4	0.517182	0.155956	0.026527	0.1701	3.316	3.490	−5.0 %
5	0.652766	0.152881	0.025770	0.1686	4.270	4.326	−1.3 %
6	0.761233	0.158443	0.026710	0.1686	4.804	5.157	−6.8 %
7	0.899427	0.152600	0.025698	0.1684	5.894	5.990	−1.6 %
8	1.032283	0.153228	0.027133	0.1771	6.737	6.760	−0.3 %

9	1.130357	0.160921	0.026626	0.1655	7.024	7.676	−8.5 %
10	1.287582	0.159949	0.026760	0.1673	8.050	8.494	−5.2 %
11	1.425094	0.160359	0.026558	0.1656	8.887	9.344	−4.9 %
12	1.562639	0.159755	0.026941	0.1686	9.781	10.145	−3.6 %
13	1.685479	0.162104	0.028531	0.1760	10.398	10.888	−4.5 %
14	1.802958	0.176959	0.027089	0.1531	10.189	12.010	−15.2 %
15	1.938491	0.169202	0.027596	0.1631	11.457	12.717	−9.9 %
16	2.063817	0.230183	0.026977	0.1172	8.966	14.242	−37.0 %

s se mantiene casi constante, entre 0.165 y 0.177 (promedio 0.1648), porque tanto la parte secuencial como el pedazo de cada hilo tienen tamaño fijo. Las excepciones son p = 14 (0.153) y p = 16 (0.117): ahí t_par creció por la interferencia descrita en P1.1, pero la parte secuencial no, así que s bajó.

P4.4. Segundo modelo. En la ejecución paralela una fracción s de Tp es secuencial y (1 − s)·Tp lo hacen p hilos a la vez. En un solo núcleo, la parte paralela tardaría p veces más:

```
Ts = s·Tp + p·(1 − s)·Tp
S(s, p) = Ts/Tp = s + p·(1 − s) = p − s·(p − 1)
```

Verificación (tabla de P4.3): con el s medido, el modelo acierta dentro de −8.5 % a +2.3 % para p ≤ 13 (por ejemplo p = 8: 6.760 contra 6.737 medido). Se aleja en p = 14 (−15.2 %) y sobre todo en p = 16 (−37 %). Para p ≥ 9 el modelo queda siempre arriba: supone que el trabajo paralelo de cada hilo tardaría lo mismo en un núcleo solo, pero con SMT cada hilo corrió más lento que si tuviera el núcleo para él, así que p·(1 − s)·Tp sobreestima Ts.

P4.5. "¿Cuánto más rápido resuelvo el mismo problema?" corresponde a la Parte 1 (problema fijo, Amdahl). Ejemplo: una consulta web o el renderizado de un cuadro con un tiempo de respuesta máximo, donde el tamaño ya está dado y queremos menos latencia. "¿Qué tanto más grande puedo hacer el problema en el mismo tiempo?" corresponde a la Parte 4 (problema que crece, Gustafson). Ejemplo: el pronóstico del clima, que debe salir en la misma ventana de horas pero con más núcleos usa una malla más fina; o el entrenamiento de modelos, donde más GPUs/núcleos permiten más datos o un modelo más grande en el mismo tiempo.

6. Síntesis (obligatoria)

Parte	**Fuga dominante**	**Número medido que lo demuestra**
1 — Amdahl	η_f (parte secuencial)	fp = 0.90: S(16) = 5.621, E = 0.351, contra E = 0.825 con fp = 1.00; fp despejado = 0.8769
2 — Suma pequeña	η_c (comunicación y sincronización)	N = 16, p = 16: S = 0.0013 (747× más lento); t_par = 2.240 µs contra t_seq = 3 ns

3 — Desbalance	η_b (desbalance)	static S(16) = 7.591 (E = 0.474) contra guided 15.181 (E = 0.949); trapecio p = 4: f = 0.437, S = 2.29 vs 2.317
4 — Gustafson	η_f, diluida porque el problema crece	s ≈ 0.165 constante; S(15) = 11.457 supera el límite de Amdahl (6); E ≈ 0.80 de p = 9 a 13

η_r (trabajo redundante): ningún experimento la aísla con un número medido; no la observamos.

Lo que no encaja en ninguna de las cuatro:

- SMT / Hyper-Threading (P1.1): con fp = 1.00 pasamos de S(8) = 7.705 a S(15) = 14.004 con solo 8 núcleos físicos; los hilos SMT rinden ~0.9 de un hilo completo.
- Ancho de banda de memoria (P2.7): con N = 16M, S se estanca en ~3.6–4.1 con ~44–49 GB/s efectivos; más hilos no leen más rápido.
- Interferencia del sistema con todos los CPU lógicos ocupados: en p = 16 S bajó en la Parte 1 (14.004 → 13.203) y en la Parte 4 (11.457 → 8.966). Hipótesis no medida; se parece a η_b pero viene de fuera del programa.
- Frecuencia/turbo y virtualización WSL2: posibles causas de variación que no medimos.
- Variabilidad de ±2 % entre t_seq y t_par con p = 1 (P2.1), que limita qué diferencias pequeñas son significativas.

7. Reto opcional — ¿speedup superlineal?

Cambio en lab12_speedup.c: solo el arreglo tamanos[] de exp_suma, de {16, 1024, 65536, 1048576, 16777216} a {65536, 131072, 262144, 524288, 1048576, 2097152, 4194304} (0.5 a 32 MiB). Elegimos tamaños alrededor de las cachés reales: L2 de 1 MiB por núcleo y L3 de 16 MiB. La idea de la pista: un arreglo que no cabe en la L2 de un núcleo (el secuencial lo lee de L3) pero cuyo pedazo por hilo sí cabe en la L2 de cada núcleo (por ejemplo 2–4 MiB con p = 8). Hicimos 5 corridas independientes de ./lab12 suma con el binario modificado; los 10 CSV oficiales se midieron antes con el código original (sha256 registrado en la evidencia).

N (MiB)	**S mediana p=8**	**mín–máx p=8**	**S mediana p=16**	**S > p en las 5 corridas**
0.5	6.373	5.266–6.528	7.020	no
1	7.081	6.277–7.238	10.694	no
2	7.134	6.961–7.236	8.617	no
4	6.998	6.083–7.598	2.688	no
8	7.539	7.379–7.758	11.762	no
16	7.128	6.212–7.600	4.979	no
32	6.228	5.230–6.820	4.864	no

Resultado: no logramos un speedup superlineal repetible. Con p ≥ 2, ningún par (N, p) tuvo S > p en las 5 corridas; algunos lo tuvieron en 1 o 2 de 5 (por ejemplo p = 2, N = 4 MiB, máximo 2.144), lo que es ruido y no cuenta. El único caso con S > p en las 5 fue N = 8 MiB con p = 1 (mediana 1.029): es un S > p falso, porque con un solo hilo la caché disponible es la misma; es la diferencia de ~3 % entre la versión OpenMP y la secuencial que ya vimos en P2.1.

Por qué no aparece: el costo secuencial por elemento fue prácticamente el mismo en todos los tamaños, entre 0.655 y 0.673 ns, tanto con 0.5 MiB (en L2) como con 32 MiB (en RAM). La suma secuencial no está limitada por la memoria sino por la cadena de sumas dependientes (cada s += a[i] espera a la anterior), así que meter los datos en una caché más rápida no la acelera y no hay ganancia extra que repartir. Con p = 16 la variación entre corridas fue enorme (por ejemplo 0.372–9.211 con 0.5 MiB), consistente con la interferencia de usar todos los CPU lógicos.

8. Archivos entregados

- reporte_lab12.pdf (este documento) y evidencia_lab12.pdf (salidas de lscpu, make, ejecución de los 4 modos, reto y las 7 gráficas).
- resultados_lab12.zip: 10 CSV originales, 4 CSV *_con_S_E.csv, 7 PNG, CSV del reto y copia de ambos PDF.
- lab12_speedup.c con el cambio del reto en tamanos[].